网络安全技术第三次作业：基于MD5算法的文件完整性校验程序

邹佳衡（2312322）

- 网络安全技术第三次作业：基于MD5算法的文件完整性校验程序
  - 实验环境
  - 实验要求
  - 实验原理
    - MD5算法
  - 代码实现
    - 程序文件结构
    - 核心数据结构
    - 核心函数说明
    - 程序完整流程
      - 1. 程序整体流程图
      - 2. MD5 算法核心流程图
      - 3. 文件完整性校验流程图
    - 编译与运行
  - 实验结果

实验环境

Ubuntu 18.04，VMware WorkStation 17

实验要求

MD5算法是目前最流行的一种信息摘要算法，在数字签名，加密与解密技术，以及文件完整性检测等领域中发挥着巨大的作用。熟悉MD5算法对开发网络应用程序，理解网络安全的概念具有十分重要的意义。本章编程训练的目的如下：

- ①深入理解MD5算法的基本原理。
- ②掌握利用MD5算法生成数据摘要的所有计算过程。
- ③掌握Linux系统中检测文件完整性的基本方法。
- ④熟悉Linux系统中文件的基本操作方法。

本章编程训练的要求如下：

- ①准确地实现MD5算法的完整计算过程。
- ②对于任意长度的字符串能够生成128位MD5摘要。
- ③对于任意大小的文件能够生成128位MD5摘要。
- ④通过检查MD5摘要的正确性来检验原文件的完整性。

在Linux平台下编写应用程序，正确地实现MD5算法。要求程序不仅能够为任意长度的字符串生成MD5摘要，而且可以为任意大小的文件生成MD5摘要。同时，程序还可以利用MD5摘要验证文件的完整性。验证文件完整性分为两种方式：一种是在手动输入MD5摘要的条件下，计算出当前被测文件的MD5摘要，再将两者进行比对。若相同，则文件完好；否则，文件遭到破坏。另一种是先利用Linux系统工具md5sum为被测文件生成一个

后缀为.md5的同名文件，然后让程序计算出被测文件的MD5摘要，将其与.md5文件中的MD5摘要进行比较，最后得出检测结果。

程序为命令行程序，可执行文件名为MD5.exe，命令行格式如下：

```
./MD5 [选项] [被测文件路径] [.md5文件路径]
```

其中 [选项] 是程序为用户提供的各种功能。在本程序中 [选项] 包括 {-h,-t,-c,-v,-f} 5个基本功能。[被测文件路径] 为应用程序指明被测文件在文件系统中的路径。[.md5文件路径] 为 应用程序指明由被测文件生成的.md5文件在文件系统中的路径。其中前两项为必选项，后两项可以根据功能进行选择。

1.打印帮助信息

在控制台命令行中输入./MD5 -h，打印程序的帮助信息。

```
[root@localhost MD5]# ./MD5 -h
MD5: usage: [-h]    --help information
            [-t]    --test MD5 application
            [-c]    [file path of the file computed]
                    --compute MD5 of the given file
            [-v]    [file path of the file validated]
                    --validate the integrality of a given file by manual input MD5
value
            [-f]    [file path of the file validated]    [file path of the .md5
file]
                    --validate the integrality of a given file by read MD5 value
from .md5file
```

2.打印测试信息

在控制台命令行中输入./MD5 -t，打印程序的测试信息。如下所示，测试信息是指本程序对特定字符串输入所生成的MD5摘要。

```
[root@localhost MD5]# ./MD5 -t
MD5("") = d41d8cd98f00b204e9800998ecf8427e
MD5("a") = 0cc175b9c0f1b6a831c399e269772661
MD5("abc") = 900150983cd24fb0d6963f7d28e17f72
MD5("message digest") = f96b697d7cb7938d525a2f31aaf161d0
MD5("abcdefghijklmnopqrstuvwxyz") = c3fcd3d76192e4007dfb496cca67e13b
MD5("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789") =
d174ab98d277d9f5a5611c2c9f419d9f
MD5("12345678901234567890123456789012345678901234567890123456789012345678901234567
890") = 57edf4a22be3c955ac49da2e2107b67a
```

3.为指定文件生成MD5摘要

在控制台命令行中输入./MD5 -c [被测文件路径]，计算出的被测文件的MD5摘要并打印出来。

```
[root@localhost MD5]# ./MD5 -c nankai.txt
The MD5 value of file("nankai.txt") is 3c771aea5b7e191408ab6b0372ecbf0c
```

4.验证文件完整性方法一

在控制台命令行中输入./MD5 -v [被测文件路径]，程序会先让用户输入被测文件的MD5摘要，然后重新计算被测文件的MD5摘要，最后将两个摘要逐位比较。若一致，则说明文件是完整的，否则，说明文件遭到破坏。

```
[root@localhost MD5]# ./MD5 -v nankai.txt
Please input the MD5 value of
file("nankai.txt")...abcdefghijklmnopqrstuvwxyz123456
The old MD5 value of file("nankai.txt") you have input is
abcdefghijklmnopqrstuvwxyz123456
The new MD5 value of file("nankai.txt") that has computed is
3c771aea5b7e191408ab6b0372ecbf0c
Match Error! The file has been modified!
[root@localhost MD5]# ./MD5 -v nankai.txt
Please input the MD5 value of
file("nankai.txt")...3c771aea5b7e191408ab6b0372ecbf0c
The old MD5 value of file("nankai.txt") you have input is
3c771aea5b7e191408ab6b0372ecbf0c
The new MD5 value of file("nankai.txt") that has computed is
3c771aea5b7e191408ab6b0372ecbf0c
OK! The file is integrated
```

5.验证文件完整性方法二

在控制台命令行输入./MD5 -f [被测文件路径] [.md5文件路径]，程序会自动读取.md5文件中的摘要，然后重新计算出被测文件的MD5摘要，最后将两者逐位比较。若一致，则说明文件是完整的，否则，说明文件遭到破坏。

```
[root@localhost MD5]# ./MD5 -f nankai.txt nankai.md5
The old MD5 value of file("nankai.txt") in nankai.md5 is
3c771aea5b7e191408ab6b0372ecbf0c
The new MD5 value of file("nankai.txt") that has computed is
3c771aea5b7e191408ab6b0372ecbf0c
OK! The file is integrated
```

实验原理

MD5算法

MD5（message-digest algorithm 5的缩写）是一种信息摘要算法。信息摘要算法的研究由来已久，早在1990年Ronald L. Rivest就提出了与MD5属于同一系列的散列函数MD4。经过大量的密码学分析与不断的攻击检测，Rivest于1991年提出了它的改进算法MD5。目前MD5已经得到了广泛的应用，成为许多机构和组织的散列函数标准。

作为散列函数，MD5有两个重要的特性：第一、任意两组数据经过MD5运算后，生成相同摘要的概率微乎其微；第二、即便在算法与程序已知的情况下，也不能够从MD5摘要中获得原始的数据。

MD5的典型应用就是为一段信息（message）生成信息摘要（message-digest），以防止被篡改。Linux系统自带了计算和校验MD5摘要的工具程序—md5sum。用户可以在命令行终端直接运行该程序。md5sum程序可以创建一个与原始文件名称相同，后缀为.md5的文件，并将原始文件的MD5摘要保存在该文件中。

代码实现

程序文件结构

本程序由 3 个源文件组成：

文件	功能
`MD5.h`	头文件：宏定义（初始化向量、位运算、非线性函数）、MD5_CTX 结构体、所有函数声明
`MD5.cpp`	算法实现：5层函数逐步实现 MD5 的完整计算流程
`main.cpp`	命令行入口：解析 -h/-t/-c/-v/-f 参数，分发到对应功能函数

核心数据结构

```
typedef struct {
    uint32_t A;             // 寄存器 A，初始值 0x67452301
    uint32_t B;             // 寄存器 B，初始值 0xEFCDAB89
    uint32_t C;             // 寄存器 C，初始值 0x98BADCFE
    uint32_t D;             // 寄存器 D，初始值 0x10325476
    uint32_t count[2];      // 原始消息比特长度（64位，低32位在前）
    uint8_t  buffer[64];    // 512位消息分组缓冲区
    uint8_t  digest[16];    // 最终128位摘要输出
} MD5_CTX;
```

核心函数说明

基础转换函数

函数	功能	输入	输出
`decodeToUint32()`	按小端序将64字节块解码为16个32位字	`block[64]`	`X[16]`
`encodeFromUint32()`	按小端序将4个32位寄存器编码为16字节	`state[4]`	`output[16]`
`digestToHexString()`	将16字节摘要转为32字符十六进制字符串	`digest[16]`	`string`

MD5 核心变换

函数	功能
`md5Transform()`	对单个512位分组执行4轮×16步非线性变换，完成后将中间结果累加回寄存器

该函数内部使用局部 STEP 宏封装每一步运算：

$$a = b + ROTATE_LEFT(a + f(b,c,d) + X[k] + T[i], s)$$

4轮分别使用不同的非线性函数：

- 第1轮 $F(X,Y,Z) = (X \wedge Y) \vee (\neg X \wedge Z)$
- 第2轮 $G(X,Y,Z) = (X \wedge Z) \vee (Y \wedge \neg Z)$
- 第3轮 $H(X,Y,Z) = X \oplus Y \oplus Z$
- 第4轮 $I(X,Y,Z) = Y \oplus (X \vee \neg Z)$

MD5 核心三步

函数	功能
`md5Init()`	初始化上下文：设置 A/B/C/D 初始向量，清零计数器
`md5Update()`	喂入数据：将数据累积到内部缓冲区，每攒满64字节调用一次 md5Transform
`md5Final()`	最后处理，包括填充，附加原始信息，数据处理

高层接口

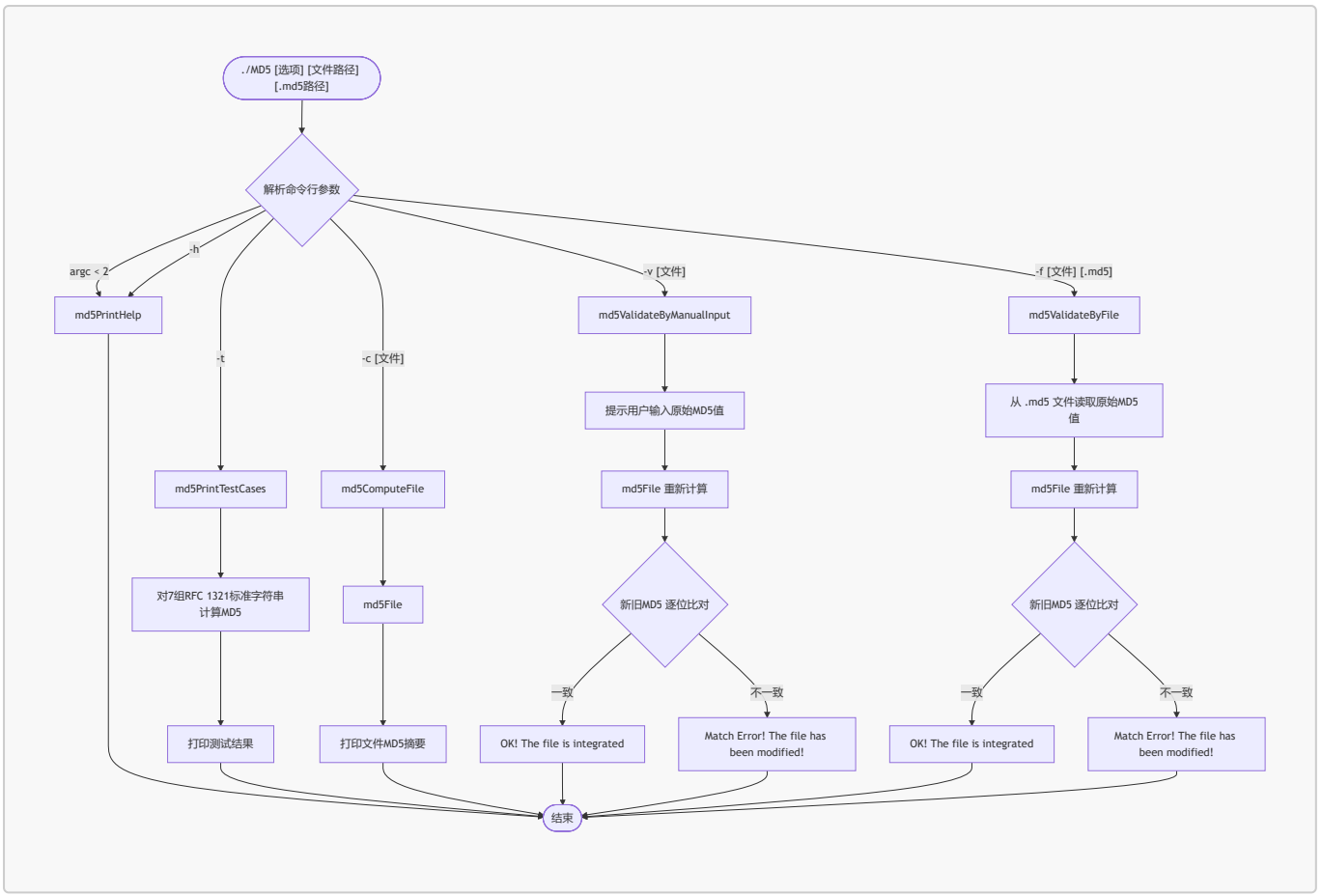
函数	功能
`md5String()`	计算任意长度字符串的 MD5 摘要。
`md5File()`	计算任意大小文件的 MD5 摘要，以4KB为单位分块读取文件，避免大文件撑爆内存，其余流程同 `md5String`

命令行与校验

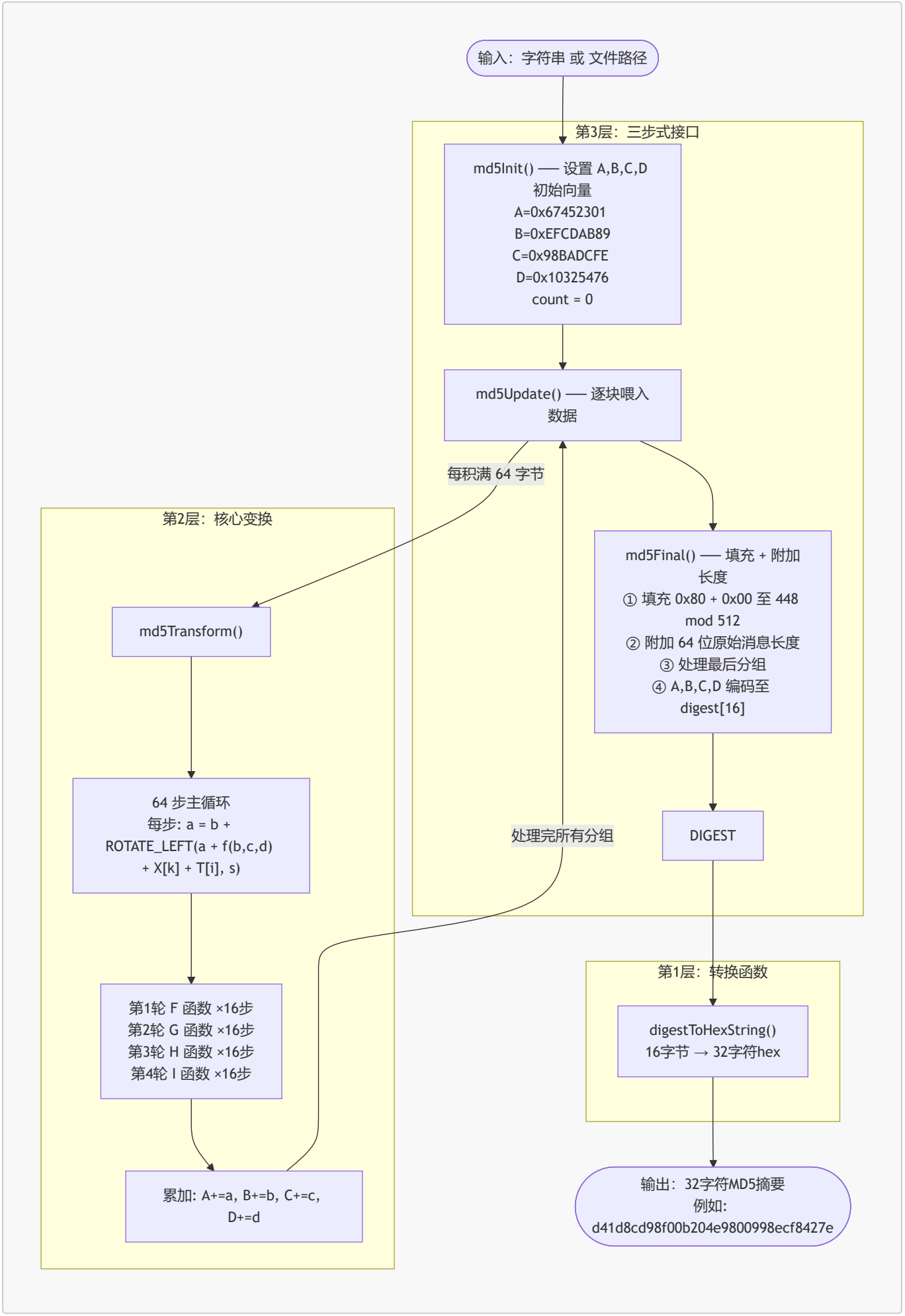
函数	功能	对应选项
`md5PrintHelp()`	打印帮助信息	`-h`
`md5PrintTestCases()`	输出测试量的 MD5 值	`-t`
`md5ComputeFile()`	计算并打印指定文件的 MD5 摘要	`-c`
`md5ValidateByManualInput()`	用户手动输入 MD5，程序重新计算，逐位比对	`-v`
`md5ValidateByFile()`	从 .md5 文件读取 MD5，程序重新计算，逐位比对	`-f`

程序完整流程

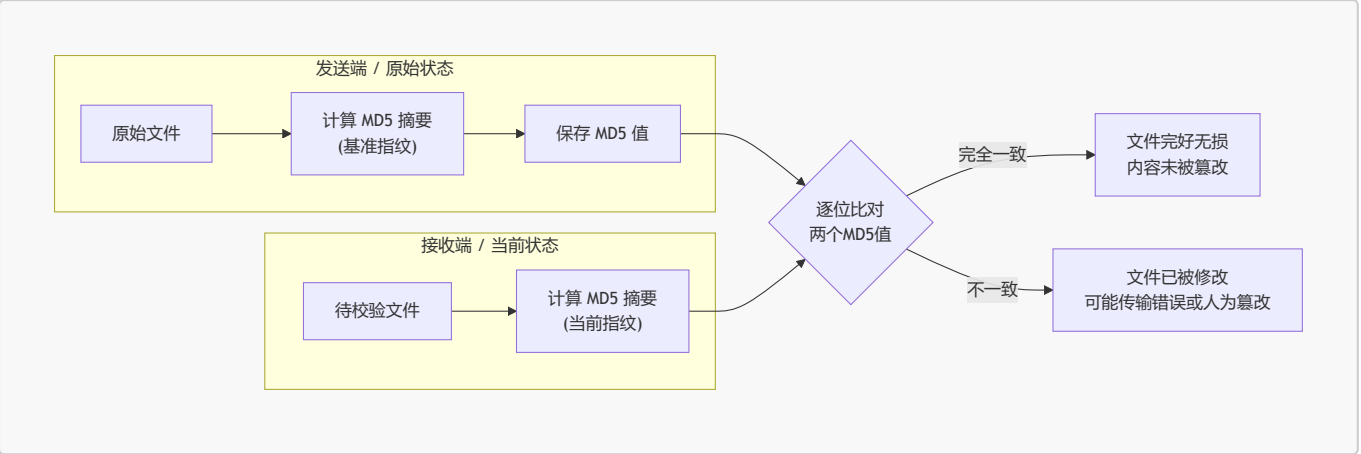
1. 程序整体流程图



2. MD5 算法核心流程图


输入：字符串 或 文件路径
第3层：三步式接口
md5Init() —— 设置 A,B,C,D 初始向量
A=0x67452301
B=0xEFCDAB89
C=0x98BADCFE
D=0x10325476
count = 0
md5Update() —— 逐块喂入数据
每积满 64 字节
第2层：核心变换
md5Transform()
64 步主循环
每步: a = b + ROTATE_LEFT(a + f(b,c,d) + X[k] + T[i], s)
第1轮 F 函数 ×16步
第2轮 G 函数 ×16步
第3轮 H 函数 ×16步
第4轮 I 函数 ×16步
累加: A+=a, B+=b, C+=c, D+=d
处理完所有分组
md5Final() —— 填充 + 附加长度
① 填充 0x80 + 0x00 至 448 mod 512
② 附加 64 位原始消息长度
③ 处理最后分组
④ A,B,C,D 编码至 digest[16]
DIGEST
第1层：转换函数
digestToHexString()
16字节 → 32字符hex
输出：32字符MD5摘要
例如:
d41d8cd98f00b204e9800998ecf8427e

3. 文件完整性校验流程图



编译与运行

```
# 编译主程序
g++ -std=c++14 -Wall -o MD5 main.cpp MD5.cpp

# 使用示例
./MD5 -h                    # 打印帮助信息
./MD5 -t                    # 打印 RFC 1321 测试向量
./MD5 -c nankai.txt         # 计算文件 MD5 摘要
./MD5 -v nankai.txt         # 手动输入 MD5 验证文件完整性
./MD5 -f nankai.txt nankai.md5  # 从 .md5 文件验证文件完整性
```

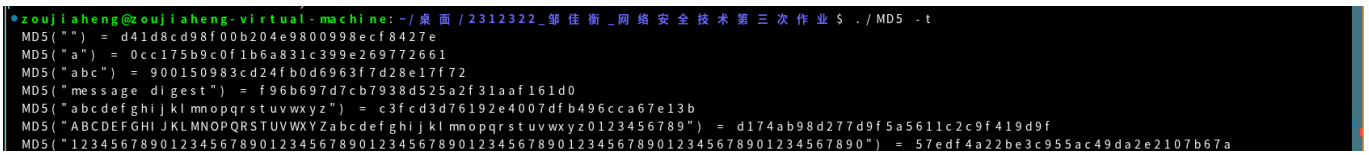
实验结果

1.输出帮助信息



程序能够正常输出帮助信息

2.输出测试信息



程序能够正常输出测试信息

3.计算文件的数据摘要



我们将计算出的摘要保存下来

4.通过摘要比对文件



第一次测试时使用原文件和正确的摘要，显示匹配正确；

第二次测试时使用原文件和错误的摘要，显示匹配错误；

第三次测试时使用更改过的文件和正确的摘要，显示匹配错误。

5.通过MD5文件比对



能够正常通过MD5文件进行校验。